Mrp Rotation

Component Design Document

1 Description

MRP rotation attitude guidance. Applies a fixed body correction, a pointing offset, and a constant rate rotation to an input attitude reference frame, advancing the rotation by one control period each tick, and returns the resulting reference attitude, rate, and acceleration. Wraps the MrpRotationAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 3
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - *None*
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 3
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - *None*
- **Number of Data Dependencies** - *None*
- **Number of Packets** - *None*

3.2 Diagram

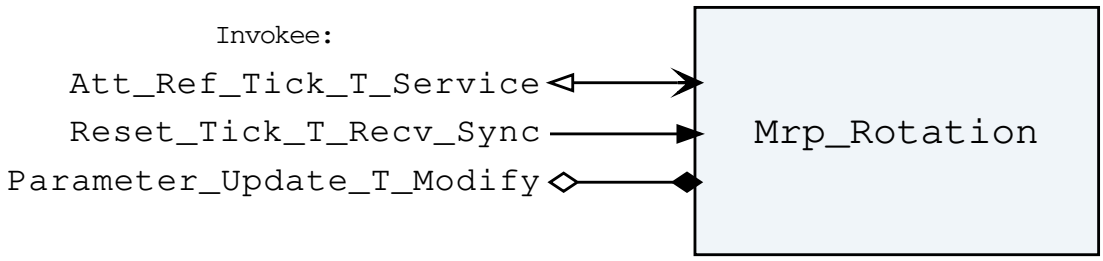


Figure 1: Mrp Rotation component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Mrp Rotation Invokee Connectors

Name	Kind	Type	Return_Type	Count
Att_Ref_Tick_T_Service	service	Att_Ref_Tick.T	Att_Ref.T	1
Reset_Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Parameter_Update_T_Modify	modify	Parameter_Update.T	-	1

Connector Descriptions:

- **Att_Ref_Tick_T_Service** - Rotate the attitude reference in the argument by the configured rotation, advanced to the tick in the argument, and return the result.
- **Reset_Tick_T_Recv_Sync** - Restart the integrated rotation at identity. Called on GNC state change so the rotation does not carry over from the previous state.
- **Parameter_Update_T_Modify** - The parameter update connector.

3.3.2 Invoker Connectors

None

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 2: Mrp Rotation Set Id Bases Parameters

Name	Type
`Parameter_Id_Base`	`Parameter_Types.Parameter_Id_Base`

Parameter Descriptions:

- **`Parameter_Id_Base`** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains no data dependencies.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the MRP rotation algorithm with the control period and the default parameter values. The `init` subprogram requires the following parameters:

Table 3: Mrp Rotation Implementation Initialization Parameters

Name	Type	Default Value
`Control_Period`	`Basic_Types.Positive_ Short_Float`	*None provided*

Parameter Descriptions:

- **`Control_Period`** - [s] Time between two algorithm updates. The rotation advances by this much per tick, so it must equal the rate at which the component is scheduled. Fixed by the assembly, so it is not a runtime parameter. The type keeps it greater than zero.

3.6 Commands

The Mrp Rotation component has no commands.

3.7 Parameters

Parameters for the Mrp Rotation component.

Table 4: Mrp Rotation Parameters

Local ID	Parameter Name	Type	Default Value
`0x0000 (0)`	`Sigma_Rcr0`	`Packed_F32x3.T`	`[0.0,0.0,0.0]`
`0x0001 (1)`	`Sigma_Rprc`	`Packed_F32x3.T`	`[0.0,0.0,0.0]`

0x0002 (2)	Omega_Rrp_R	Packed_F32x3.T	[0.0,0.0,0.0]

Parameter Descriptions:

- **Sigma_Rcr0** - [-] Fixed body correction, the MRP of the corrected frame Rc relative to the input reference R0. It sets which body axes align with the input reference axes. Must be finite.
- **Sigma_Rprc** - [-] Pointing offset, the MRP of the frame Rp relative to Rc in body axes. It tilts the corrected frame, for example off the target. Must be finite.
- **Omega_Rrp_R** - [rad/s] Constant rotation rate of the output frame R relative to Rp in output (body) frame components. The rotation is integrated from identity. Must be finite.

3.8 Events

The Mrp Rotation component has no events.

3.9 Data Products

The Mrp Rotation component has no data products.

3.10 Data Dependencies

The Mrp Rotation component has no data dependencies.

3.11 Packets

The Mrp Rotation component has no packets.

3.12 Faults

The Mrp Rotation component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Mrp_Rotation_Tests* Test Suite

This is a unit test suite for the Mrp Rotation component

Test Descriptions:

- **Test** - Run the algorithm through a rotation, pointing offset change, and reset sequence built from the cases of the Python reference test, with expected values from its reference model, to ensure the Ada to C to C++ integration is sound.
- **Test_Invalid_Parameter** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **Test_Reset_Applies_Pending_Parameters** - Ensure a reset applies a configuration that was updated but not yet ticked, so the rotation restarts at identity with the new pointing offset.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Att_Ref.T:

Table 5: Att_Ref Packed Record : 288 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sigma_Rn	Packed_F32x3.T	-	96	0	95
Omega_Rn_N	Packed_F32x3.T	-	96	96	191
Domega_Rn_N	Packed_F32x3.T	-	96	192	287

Field Descriptions:

- **Sigma_Rn** - *No description provided.*
- **Omega_Rn_N** - *No description provided.*
- **Domega_Rn_N** - *No description provided.*

Att_Ref_Tick.T:

The algorithm tick a guidance stage runs on and the attitude reference it starts from. The component that chains the guidance stages hands this to a stage and receives the stage's reference back.

Table 6: Att_Ref_Tick Packed Record : 432 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Current_Tick	Tick.T	-	80	0	79
Call_Time	Interfaces. Unsigned_64	0 to 18446744073709551615	64	80	143
Reference	Att_Ref.T	-	288	144	431

Field Descriptions:

- **Current_Tick** - The tick as scheduled, which the stage uses as the stale reference for its data dependencies.
- **Call_Time** - [ns] The time the stage propagates to, in nanoseconds of the tick time, as the algorithm tick carries it.
- **Reference** - The attitude reference the stage starts from.

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 7: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 8: Parameter Packed Record : 792 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_ Header.T	-	24	0	23	-
Buffer	Parameter_ Types. Parameter_ Buffer_Type	-	768	24	791	Header.Buffer_ Length

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 9: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_ Length_Type	0 to 96	8	16	23

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 10: Parameter_Update Packed Record : 824 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length

Table_Id	Parameter_Types.Parameter_Table_Id	0 to 65535	16	0	15	-
Operation	Parameter_Enums.Parameter_Operation_Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate 4 => Discard	8	16	23	-
Status	Parameter_Enums.Parameter_Update_Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter.T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 11: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces.Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces.Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 12: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 13: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.
Discard	4	Discard any parameters staged by this table that have not been updated.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 14: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.